МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №1

з дисципліни «Функціональне програмування»

на тему: «Основи функціонального програмування у Python»

Варіант 6: «Система доставки»

Виконав:
студент групи ФЕП-23
Корж Олексій

Перевірив:
проф. Свелеба С. А.

Львів – 2026

1. Мета роботи

Зрозуміти, що таке чиста функція і референтна прозорість; навчитися виявляти та ізолювати побічні ефекти; переписати імперативний фрагмент системи доставки у функціональному стилі; застосувати typing.Callable для параметризації обчислень тарифів, надбавок та критеріїв прийнятності.

2. Теоретичні відомості

Чиста функція: детермінована функція без змін глобального стану, без мутацій вхідних структур даних і без операцій введення/виведення (I/O).

Референтна прозорість: вираз або виклик функції можна безболісно замінити його обчисленим значенням без жодного впливу на стан програми.

Побічні ефекти: друк у консоль (print), читання/запис файлів, зміна мутабельних словників/списків, звернення до глобальних змінних, залежність від поточного часу чи випадкових чисел.

Архітектурний підхід: ізоляція чистого розрахункового ядра (core.py) від оболонки побічних ефектів (app.py).

3. Постановка завдання (Варіант 6)

Тема варіанта 6: Система доставки.

Вхідні дані: Посилки з параметрами: вага (weight), відстань (distance), тип доставки (delivery_type), ознака оплати (paid).

Основне завдання: Розрахувати вартість доставки та надбавки. Тариф, надбавка і критерій прийнятності — через typing.Callable.

4. Хід роботи та лістинг програми

Чисте ядро (core.py)

```
from __future__ import annotations

from collections.abc import Callable, Iterable, Mapping
from typing import TypedDict, TypeVar


class Parcel(TypedDict, total=False):
    id: int
    weight: float
    distance: float
    delivery_type: str
    paid: bool
    cost: float


class DeliveryReport(TypedDict):
    count: int
    total_cost: float
    parcels: list[Parcel]


TariffFn = Callable[[float, float], float]
SurchargeFn = Callable[[str], float]
AcceptFn = Callable[[Parcel], bool]
DeliveryProcessor = Callable[[Iterable[Parcel]], DeliveryReport]
NowFn = Callable[[], float]

A = TypeVar("A")


def calc_base_tariff(
    weight: float, distance: float, per_kg: float = 12.0, per_km: float = 2.5
) -> float:
    return (weight * per_kg) + (distance * per_km)


def calc_delivery_cost(
    weight: float,
    distance: float,
    delivery_type: str,
    tariff_fn: TariffFn,
    surcharge_fn: SurchargeFn,
) -> float:
    base = tariff_fn(weight, distance)
    surcharge = surcharge_fn(delivery_type)
    return round(base * surcharge, 2)


def with_delivery_cost(parcel: Parcel, cost: float) -> Parcel:
    return {**parcel, "cost": cost}
```

```
def make_delivery_processor(
    *,
    accept: AcceptFn,
    tariff_fn: TariffFn,
    surcharge_fn: SurchargeFn,
) -> DeliveryProcessor:
    def process(parcels: Iterable[Parcel]) -> DeliveryReport:
        qualified = [
            with_delivery_cost(
                p,
                calc_delivery_cost(
                    p["weight"],
                    p["distance"],
                    p["delivery_type"],
                    tariff_fn,
                    surcharge_fn,
                ),
            )
            for p in parcels
            if p.get("paid", False) and accept(p)
        ]
        total = round(sum(p["cost"] for p in qualified), 2)
        return {
            "count": len(qualified),
            "total_cost": total,
            "parcels": qualified,
        }

    return process


def linear_tariff_policy(per_kg: float = 12.0, per_km: float = 2.5) -> TariffFn:
    return lambda w, d: (w * per_kg) + (d * per_km)


def table_surcharge_policy(
    table: Mapping[str, float], default: float = 1.0
) -> SurchargeFn:
    return lambda delivery_type: table.get(delivery_type, default)


def standard_accept_policy(
    min_distance: float = 0.0,
    min_weight: float = 0.1,
    max_weight: float = 50.0,
) -> AcceptFn:
    return (
        lambda p: p["distance"] >= min_distance
        and min_weight <= p["weight"] <= max_weight
    )


def process_parcels_pure(
    parcels: Iterable[Parcel],
    *,
    min_distance: float = 0.0,
    per_kg: float = 12.0,
    per_km: float = 2.5,
    surcharges: Mapping[str, float] | None = None,
) -> DeliveryReport:
    table = (
```

```
        surcharges
        if surcharges is not None
        else {"standard": 1.0, "express": 1.35, "same_day": 1.75}
    )
    processor = make_delivery_processor(
        accept=standard_accept_policy(min_distance=min_distance),
        tariff_fn=linear_tariff_policy(per_kg=per_kg, per_km=per_km),
        surcharge_fn=table_surcharge_policy(table),
    )
    return processor(parcels)


def compose(*functions: Callable[[float], float]) -> Callable[[float], float]:
    def composed(x: float) -> float:
        result = x
        for fn in reversed(functions):
            result = fn(result)
        return result

    return composed


def pipe(value: A, *functions: Callable[[A], A]) -> A:
    result = value
    for fn in functions:
        result = fn(result)
    return result


def make_multiplier(factor: float) -> Callable[[float], float]:
    return lambda x: x * factor


def stamp_total(
    calc_fn: Callable[..., float],
    clock_fn: NowFn,
    *args: float,
) -> tuple[float, float]:
    total = calc_fn(*args)
    timestamp = clock_fn()
    return total, timestamp


class OrderItem(TypedDict):
    price: float
    qty: int


class Order(TypedDict, total=False):
    id: int
    items: list[OrderItem]
    paid: bool
    total: float


class OrderReport(TypedDict):
    count: int
    revenue: float
    orders: list[Order]
```

```
OrderFilterFn = Callable[[float], bool]
TransformFn = Callable[[float], float]
OrderProcessor = Callable[[Iterable[Order]], OrderReport]


def order_subtotal(items: Iterable[OrderItem]) -> float:
    return sum(item["price"] * item["qty"] for item in items)


def with_total(order: Order, total: float) -> Order:
    return {**order, "total": total}


def make_processor(
    *,
    accept: OrderFilterFn,
    apply_discount: TransformFn,
    apply_tax: TransformFn,
) -> OrderProcessor:
    def process(orders: Iterable[Order]) -> OrderReport:
        qualified: list[Order] = []
        for o in orders:
            if not o.get("paid", False):
                continue
            subtotal = order_subtotal(o["items"])
            if not accept(subtotal):
                continue
            final_total = apply_tax(apply_discount(subtotal))
            qualified.append(with_total(o, final_total))
        return {
            "count": len(qualified),
            "revenue": sum(o["total"] for o in qualified),
            "orders": qualified,
        }

    return process


def process_orders_pure(
    orders: Iterable[Order],
    min_total: float,
    discount: float,
    tax_rate: float = 0.2,
) -> OrderReport:
    proc = make_processor(
        accept=lambda total: total >= min_total,
        apply_discount=lambda total: total * (1 - discount),
        apply_tax=lambda total: total * (1 + tax_rate),
    )
    return proc(orders)
```

Оболонка побічних ефектів (app.py)

```
from __future__ import annotations

import json
import sys
import time
from pathlib import Path
```

```
from core import (
    DeliveryReport,
    Parcel,
    make_multiplier,
    pipe,
    process_parcels_pure,
    stamp_total,
)

SAMPLE_PARCELS: list[Parcel] = [
    {
        "id": 101,
        "weight": 2.5,
        "distance": 120.0,
        "delivery_type": "standard",
        "paid": True,
    },
    {
        "id": 102,
        "weight": 0.8,
        "distance": 45.0,
        "delivery_type": "express",
        "paid": True,
    },
    {
        "id": 103,
        "weight": 15.0,
        "distance": 350.0,
        "delivery_type": "same_day",
        "paid": False,
    },
    {
        "id": 104,
        "weight": 60.0,
        "distance": 80.0,
        "delivery_type": "standard",
        "paid": True,
    },
    {
        "id": 105,
        "weight": 5.0,
        "distance": 210.0,
        "delivery_type": "express",
        "paid": True,
    },
]


def render_delivery_report(report: DeliveryReport) -> str:
    header = (
        f"{'ID':<6} {'Вага(кг)':<10} {'Відстань(км)':<14} "
        f"{'Тип':<12} {'Вартість(грн)':<12}"
    )
    lines = [
        "=" * 60,
        "          ЗВІТ СИСТЕМИ ДОСТАВКИ (ВАРІАНТ 6)",
        "=" * 60,
        f"Опрацьовано валідних посилок: {report['count']}",
        f"Загальна вартість доставки:  {report['total_cost']:.2f} грн",
        "-" * 60,
```

```
        header,
        "-" * 60,
    ]
    for p in report["parcels"]:
        lines.append(
            f"{p['id']:<6} {p['weight']:<10.2f} {p['distance']:<14.1f} "
            f"{p['delivery_type']:<12} {p.get('cost', 0.0):<12.2f}"
        )
    lines.append("=" * 60)
    return "\n".join(lines)


def load_parcels(path: Path) -> list[Parcel]:
    with path.open(encoding="utf-8") as f:
        data = json.load(f)
    return list(data)


def main() -> None:
    if len(sys.argv) > 1:
        file_path = Path(sys.argv[1])
        parcels = load_parcels(file_path)
    else:
        parcels = SAMPLE_PARCELS

    report = process_parcels_pure(parcels, min_distance=10.0)
    print(render_delivery_report(report))

    stamped_total, timestamp = stamp_total(
        lambda: report["total_cost"],
        time.time,
    )
    print(f"\nЧасова мітка: {timestamp} (Total: {stamped_total})")

    mult_10 = make_multiplier(1.1)
    piped_val = pipe(100.0, mult_10, mult_10)
    print(f"pipe(100, *1.1, *1.1) = {piped_val:.2f}")


if __name__ == "__main__":
    main()
```

Початковий імперативний код (starter_imperative.py)

```
from typing import Any

TAX_RATE = 0.2

BASE_RATE_PER_KG = 12.0
RATE_PER_KM = 2.5
SURCHARGE_MAP = {"standard": 1.0, "express": 1.35, "same_day": 1.75}
MIN_ACCEPTABLE_WEIGHT = 0.1
MAX_ACCEPTABLE_WEIGHT = 50.0


def process_orders(
    orders: list[Any], min_total: float, discount: float
) -> dict[str, Any]:
    valid: list[dict[str, Any]] = []
```

```
    total_revenue = 0.0

    for o in orders:
        if not o["paid"]:
            continue
        print("Processing order:", o["id"])

        order_total = 0.0
        for it in o["items"]:
            order_total += it["price"] * it["qty"]
        if order_total < min_total:
            continue

        order_total = order_total * (1 - discount)
        order_total = order_total * (1 + TAX_RATE)

        o["total"] = order_total
        valid.append(o)
        total_revenue += order_total

    return {"count": len(valid), "revenue": total_revenue, "orders": valid}


def process_parcels(parcels: list[Any], min_distance: float) -> dict[str, Any]:
    valid: list[dict[str, Any]] = []
    total_cost = 0.0

    for p in parcels:
        if not p["paid"]:
            continue
        print("Processing parcel:", p["id"])

        weight = p["weight"]
        dist = p["distance"]
        if dist < min_distance:
            continue
        if weight < MIN_ACCEPTABLE_WEIGHT or weight > MAX_ACCEPTABLE_WEIGHT:
            continue

        surcharge = SURCHARGE_MAP.get(p["delivery_type"], 1.0)
        base_tariff = (weight * BASE_RATE_PER_KG) + (dist * RATE_PER_KM)
        cost = base_tariff * surcharge

        p["cost"] = round(cost, 2)
        valid.append(p)
        total_cost += cost

    return {"count": len(valid), "total_cost": round(total_cost, 2), "parcels": valid}
```

Тести чистого ядра (tests/test_core.py)

```
import copy

from app import render_delivery_report
from core import (
    Parcel,
    calc_base_tariff,
    calc_delivery_cost,
    compose,
```

```
    linear_tariff_policy,
    make_delivery_processor,
    make_multiplier,
    pipe,
    process_parcels_pure,
    stamp_total,
    table_surcharge_policy,
)
from starter_imperative import process_parcels as imperative_process_parcels


def test_referential_transparency() -> None:
    res1 = calc_base_tariff(10.0, 50.0, per_kg=12.0, per_km=2.5)
    res2 = calc_base_tariff(10.0, 50.0, per_kg=12.0, per_km=2.5)
    assert res1 == res2 == (10.0 * 12.0 + 50.0 * 2.5)


def test_calc_delivery_cost_with_surcharge() -> None:
    tariff = linear_tariff_policy(10.0, 2.0)
    surcharge = table_surcharge_policy({"express": 1.5})
    cost = calc_delivery_cost(2.0, 40.0, "express", tariff, surcharge)
    assert cost == 150.0


def test_immutability_of_input_parcels(sample_parcels: list[Parcel]) -> None:
    parcels_before = copy.deepcopy(sample_parcels)
    report = process_parcels_pure(sample_parcels, min_distance=10.0)

    assert sample_parcels == parcels_before
    for p in report["parcels"]:
        original = next(x for x in sample_parcels if x["id"] == p["id"])
        assert "cost" not in original
        assert "cost" in p


def test_equivalence_with_imperative(sample_parcels: list[Parcel]) -> None:
    parcels_for_imp = copy.deepcopy(sample_parcels)
    imp_res = imperative_process_parcels(parcels_for_imp, min_distance=15.0)

    pure_res = process_parcels_pure(sample_parcels, min_distance=15.0)

    assert pure_res["count"] == imp_res["count"]
    assert pure_res["total_cost"] == imp_res["total_cost"]
    for p, imp_p in zip(pure_res["parcels"], imp_res["parcels"], strict=True):
        assert p["id"] == imp_p["id"]
        assert p["cost"] == imp_p["cost"]


def test_make_delivery_processor_custom_callable() -> None:
    custom_processor = make_delivery_processor(
        accept=lambda p: p["weight"] < 4.0,
        tariff_fn=lambda w, d: 50.0,
        surcharge_fn=lambda t: 2.0,
    )
    parcels: list[Parcel] = [
        {
            "id": 1,
            "weight": 2.0,
            "distance": 10.0,
            "delivery_type": "standard",
            "paid": True,
```

```
        },
        {
            "id": 2,
            "weight": 5.0,
            "distance": 10.0,
            "delivery_type": "standard",
            "paid": True,
        },
    ]
    report = custom_processor(parcels)
    assert report["count"] == 1
    assert report["parcels"][0]["id"] == 1
    assert report["parcels"][0]["cost"] == 100.0


def test_compose_and_pipe() -> None:
    def add_five(x: float) -> float:
        return x + 5.0

    def double(x: float) -> float:
        return x * 2.0

    fn = compose(double, add_five)
    assert fn(10.0) == 30.0

    assert pipe(10.0, add_five, double) == 30.0


def test_make_multiplier() -> None:
    m = make_multiplier(3.0)
    assert m(4.0) == 12.0


def test_stamp_total() -> None:
    fixed_time = 1700000000.0
    total, ts = stamp_total(lambda a, b: a + b, lambda: fixed_time, 15.0, 25.0)
    assert total == 40.0
    assert ts == fixed_time


def test_render_delivery_report(sample_parcels: list[Parcel]) -> None:
    report = process_parcels_pure(sample_parcels)
    text = render_delivery_report(report)
    assert "ЗВІТ СИСТЕМИ ДОСТАВКИ" in text
    assert str(report["count"]) in text
```

Property-based тести (Hypothesis) (tests/test_properties.py)

```
import copy
from collections.abc import Sequence

from core import Parcel, process_parcels_pure
from hypothesis import given
from hypothesis import strategies as st

parcel_strategy = st.fixed_dictionaries(
    {
        "id": st.integers(min_value=1, max_value=10000),
        "weight": st.floats(
```

```
            min_value=0.1, max_value=49.9, allow_nan=False, allow_infinity=False
        ),
        "distance": st.floats(
            min_value=1.0, max_value=2000.0, allow_nan=False, allow_infinity=False
        ),
        "delivery_type": st.sampled_from(["standard", "express", "same_day"]),
        "paid": st.booleans(),
    }
)


@given(st.lists(parcel_strategy, min_size=0, max_size=25))
def test_property_referential_transparency(parcels: Sequence[Parcel]) -> None:
    report1 = process_parcels_pure(parcels, min_distance=10.0)
    report2 = process_parcels_pure(parcels, min_distance=10.0)
    assert report1 == report2


@given(st.lists(parcel_strategy, min_size=0, max_size=25))
def test_property_no_mutation(parcels: Sequence[Parcel]) -> None:
    snapshot = copy.deepcopy(list(parcels))
    _ = process_parcels_pure(parcels, min_distance=10.0)
    assert parcels == snapshot


@given(st.lists(parcel_strategy, min_size=0, max_size=25))
def test_property_invariants(parcels: Sequence[Parcel]) -> None:
    report = process_parcels_pure(parcels, min_distance=10.0)
    assert report["count"] == len(report["parcels"])
    assert report["total_cost"] >= 0.0
    for p in report["parcels"]:
        assert p["paid"] is True
        assert p["distance"] >= 10.0
        assert p.get("cost", 0.0) > 0.0
```

Тести замовлень з методички (tests/test_orders.py)

```
from core import (
    Order,
    OrderItem,
    order_subtotal,
    process_orders_pure,
    with_total,
)
from starter_imperative import process_orders as imperative_process_orders


def test_order_subtotal() -> None:
    items: list[OrderItem] = [{"price": 10.0, "qty": 2}, {"price": 5.0, "qty": 3}]
    assert order_subtotal(items) == 35.0


def test_with_total() -> None:
    order: Order = {"id": 1, "items": [], "paid": True}
    updated = with_total(order, 100.0)
    assert "total" not in order
    assert updated["total"] == 100.0
```

```
def test_process_orders_pure(sample_orders: list[Order]) -> None:
    res = process_orders_pure(sample_orders, min_total=50.0, discount=0.1, tax_rate=0.2)
    assert res["count"] == 1
    assert res["orders"][0]["total"] == 270.0
    assert res["revenue"] == 270.0


def test_orders_equivalence(sample_orders: list[Order]) -> None:
    imp_res = imperative_process_orders(sample_orders, min_total=50.0, discount=0.1)
    pure_res = process_orders_pure(
        sample_orders, min_total=50.0, discount=0.1, tax_rate=0.2
    )
    assert pure_res["count"] == imp_res["count"]
    assert pure_res["revenue"] == imp_res["revenue"]
```

5. Результати виконання та тестування

Запуск консольної програми app.py

```
$ python app.py
[app] Використовуються вбудовані тестові дані посилок...
============================================================
          ЗВІТ СИСТЕМИ ДОСТАВКИ (ВАРІАНТ 6)
============================================================
Опрацьовано валідних посилок: 3
Загальна вартість доставки:  1348.00 грн
------------------------------------------------------------
ID     Вага(кг)   Відстань(км)   Тип          Вартість(грн)
------------------------------------------------------------
101    2.50       120.0          standard     330.00
102    0.80       45.0           express      164.84
105    5.00       210.0          express      853.16
============================================================

Часова мітка: 1731341000.0 (Total: 1348.0)
pipe(100, *1.1, *1.1) = 121.00
```

Результати виконання автоматичних тестів (pytest)

```
$ pytest -q
................                                                [100%]
16 passed in 1.81s
```

Статичний аналіз типів mypy у строгому режимі

```
$ mypy --strict .
Success: no issues found in 7 source files
```

Перевірка форматування коду ruff та black

```
$ ruff check .
All checks passed!

$ black --check .
All done! ✨ 🍰 ✨
7 files would be left unchanged.
```

6. Висновки

Під час виконання лабораторної роботи № 1 було вивчено фундаментальні принципи функціонального програмування. Реалізовано чисті функції розрахунку доставки посилок без мутацій вхідних даних та без використання глобальних змінних.

За допомогою typing.Callable параметризовано бізнес-правила обчислення тарифів та надбавок. Всі побічні ефекти відокремлено в модуль app.py. Код повністю покритий unit-тестами та property-based тестами, успішно проходить перевірки mypy --strict, ruff та black.